SCOPE OF WORK

Data Intelligence Platform

Automated Data Collection, Analytics & AI-Driven Insights Platform

0. Document Control & Assumptions

This Scope of Work (SOW) is Phase 2 of the project and serves as the blueprint for the Data Intelligence Platform described in the Phase 1 project presentation. It is written to be detailed enough for a development team to execute the project without needing further clarification.

0.1 Open Items Requiring Sign-Off Before Phase 3

Two inputs were not finalized at the time of writing. Placeholders are used below and must be confirmed by the project sponsor before Requirements Gathering (Phase 3) begins:

- Designated Data Source: the specific website/portal/API to be scraped is marked as [DATA SOURCE – TBD]. All functional requirements, schema design, and legal/compliance notes below assume a single structured or semi-structured public website updated at least hourly.
- Team Size & Timeline: confirmed as a 2-person team — Rafay (Backend/.NET) and Sahil Kotak (Frontend & Database) — delivering over a 04-week schedule. Section 8 can be re-baselined if the schedule needs to change to reflect this team size.

1. Project Objectives

The primary objective is to design, build, and deploy a web-based Data Intelligence Platform that automatically collects data from a designated online source on an hourly basis, stores it durably in Microsoft SQL Server, and exposes it through analytical dashboards and a natural-language AI query assistant.

Secondary objectives (equally weighted, per project instructions) are to produce professional-grade planning, architecture, and testing artifacts, and to practice structured, phase-gated project execution.

1.1 Business Value

- Removes manual data-gathering effort by automating hourly collection.
- Turns raw web data into decision-ready KPIs and visual reports.
- Lowers the barrier to data analysis via plain-language querying (no SQL knowledge required).
- Produces a reusable reference architecture (scraper → SQL Server → .NET API → React dashboard → AI layer) for future data projects.

2. Functional Requirements

2.1 Data Collection

- FR-1: The system shall scrape/collect data from [DATA SOURCE – TBD] on an automated hourly schedule with no manual intervention.
- FR-2: The system shall detect and log collection failures (site unreachable, layout change, timeout) without crashing the scheduler.
- FR-3: The system shall deduplicate records so re-running a collection cycle does not create duplicate rows.

- FR-4: The system shall retain full historical data (no overwrite of prior snapshots) to support trend analysis.

2.2 Data Storage

- FR-5: All collected and derived data shall be persisted in Microsoft SQL Server using a normalized schema.
- FR-6: The schema shall version each record with a collection timestamp to support time-series queries.

2.3 Backend & API

- FR-7: A .NET Web API shall expose CRUD and query endpoints for dashboards and the AI assistant.
- FR-8: The backend shall run the scraper on a schedule (e.g., a hosted background/worker service) independent of user traffic.
- FR-9: The API shall implement authentication/authorization for all non-public endpoints.

2.4 Frontend / Dashboards

- FR-10: The web application (React or Next.js) shall present dashboards with KPIs, trend charts, and tabular reports.
- FR-11: Users shall be able to filter/drill down dashboards by date range and key data dimensions.
- FR-12: The UI shall be responsive across desktop, tablet, and mobile breakpoints.

2.5 AI Query Assistant

- FR-13: Users shall be able to submit a natural-language question about the data.
- FR-14: The system shall translate the question into a validated, parameterized SQL query (with safeguards against destructive statements).
- FR-15: The system shall execute the query against SQL Server and return results within an acceptable response time.
- FR-16: The system shall convert query results back into a natural-language answer for the user.

2.6 Optional: AI Visualization Engine

- FR-17 (Optional/Stretch): Users shall be able to request chart/report modifications (e.g., "show this as a bar chart by month") using natural language, with the AI generating the corresponding visualization config.

3. Non-Functional Requirements

Category	Requirement
Performance	Dashboards load in under 3 seconds for a 12-month data range under normal load.

Category	Requirement
Scalability	Schema and API support growth in data volume without redesign (partitioning-ready tables).
Reliability	Hourly collection job success rate ≥ 99% over a rolling 30-day window, with alerting on failure.
Security	Secrets (DB credentials, AI API keys) stored outside source control; role-based access on the API.
Maintainability	Layered architecture with documented API contracts and a versioned database migration history.
Usability	UI usable by a non-technical business user without training.
Auditability	All AI-generated SQL queries are logged for review before/alongside execution.
Compliance	Data collection respects the source site's terms of service and robots.txt directives.

4. System Architecture

The solution follows a five-layer architecture:

Layer	Responsibility	Key Technology
Data Collection Layer	Scheduled extraction from [DATA SOURCE – TBD]; parsing and validation before storage.	.NET Worker Service / Hosted Service + HttpClient / HtmlAgilityPack
Data Storage Layer	Structured, historized storage of raw and processed data.	Microsoft SQL Server
Backend Processing Layer	Business logic, scheduling, REST API, AI orchestration.	ASP.NET Core Web API
Frontend Application	Dashboards, KPIs, reporting, user interaction.	React or Next.js
AI Query Assistant Layer	NL-to-SQL translation and NL response generation.	LLM API (e.g., Claude/OpenAI) via backend orchestration

4.1 High-Level Data Flow

- Scheduler triggers the collector every hour → collector fetches and parses the source → validated records are written to SQL Server.
- Frontend calls the .NET API for dashboard data → API queries SQL Server → results rendered as KPIs/charts.
- User submits a natural-language question → API sends it (with schema context) to the AI service → AI returns a SQL query → API validates and executes the query → API sends the result set back to the AI service → AI returns a natural-language answer → frontend displays it.

4.2 Deployment View

- Web API and Worker Service hosted as separate deployable units (can share a host in early phases).
- SQL Server hosted as a managed or on-prem instance reachable only from the backend network segment.
- Frontend deployed as a static/SSR build served independently of the API.

5. Technology Stack

Component	Technology	Notes
Backend	.NET 8 / ASP.NET Core	Web API + Worker Service for scheduled scraping
Database	Microsoft SQL Server	Primary structured data store
ORM	Entity Framework Core	Migrations, schema versioning
Frontend	React or Next.js	Team to confirm final choice in Phase 3
Charting	A JS charting library (e.g., Chart.js/Recharts)	Dashboards, KPIs, trend charts
AI Integration	LLM API (NL-to-SQL and NL response generation)	Provider to be confirmed
Scheduling	.NET Hosted Service / Windows Task Scheduler / cron-equivalent	Hourly trigger
Source Control	Git	Feature-branch workflow
CI/CD	GitHub Actions (or equivalent)	Build, test, deploy pipeline

6. Implementation Phases & Deliverables

Phase	Key Activities	Deliverables
1. Project Presentation	Present problem, solution, value, architecture.	Approved presentation deck.
2. Scope of Work	Define requirements, architecture, plan.	This SOW document (approved).
3. Requirements & Research	Schema design, architecture finalization, feasibility.	ERD/schema, architecture doc, risk log.
4. Application Development	Build backend, DB, frontend, AI assistant.	Working application, API docs, source repo.
5. User Acceptance Testing	End-to-end testing, bug fixes, feedback.	Test plan, UAT sign-off, defect log.
6. Go-Live & Closure	Deploy, document, hand over, close out.	Deployment, user/tech docs, closure report.

7. Roles & Responsibilities

Confirmed staffing:

Role	Owner	Responsibilities
Project Sponsor / Reviewer	Jalab khan	Approves SOW and each phase gate; provides the final data source and business priorities.
.NET / Backend Developer	Rafay shahid	API, worker service, scraping logic, AI orchestration, database access layer.
Frontend Developer	Sahil Kotak	React/Next.js dashboards, UX, responsive design, chart integrations.
Database Owner	Sahil Kotak	Schema design, indexing, performance tuning, migrations.
QA / DevOps (shared role)	Rafay & Sahil Kotak	Test plans, UAT coordination, CI/CD pipeline, deployment.

8. Timeline & Milestones

Indicative 10-week schedule assuming a 2–3 person team; to be adjusted once team size and start date are confirmed.

Weeks	Phase	Milestone
Week 1	Phase 2–3	SOW approved; requirements & schema finalized.
Week2	Phase 4	Start work on development.
Week 3	Phase 4	Continue and finish development.
Week 4	Phase 5-6	UAT and Deployment

9. Risks & Mitigation

Risk	Impact	Mitigation
Target site changes layout/blocks scraping	High	Build resilient selectors, monitoring/alerts, and a manual fallback import.
Data source terms of service restrict scraping	High	Confirm source and legality before Phase 3; prefer an official API if one exists.
AI-generated SQL is inaccurate or unsafe	High	Restrict to read-only DB role, validate/parameterize queries, log every generated query.
Schema rework mid-project	Medium	Invest in schema design during Phase 3 before development starts.
Team unfamiliarity with full SDLC process	Medium	Phase-gate reviews after each deliverable; this SOW itself as reference.
Scope creep (optional AI visualization engine)	Low	Treat FR-17 as stretch scope, delivered only after core features pass UAT.

10. Acceptance Criteria

- Hourly data collection runs unattended for at least 24 consecutive hours with ≥99% success rate.
- Dashboards accurately reflect stored data and load within the performance target in Section 3.
- AI Query Assistant correctly answers at least 90% of a documented set of test questions during UAT.
- No critical or high-severity defects remain open at UAT sign-off.
- Technical and user documentation delivered and reviewed before go-live.
- Sponsor formally signs off on UAT results before production deployment.

11. Testing Strategy

11.1 Levels of Testing

- Unit Testing: backend services, data validation, and NL-to-SQL translation logic.
- Integration Testing: scraper → database → API → frontend data flow.
- System Testing: full end-to-end scenarios including AI assistant round-trips.
- User Acceptance Testing (Phase 5): business-scenario validation with the project sponsor.

11.2 Test Data & Environments

- A staging environment with a SQL Server instance seeded from real (or representative) collected data.
- A documented set of natural-language test questions with expected answers for AI assistant validation.

11.3 Defect Management

- All defects logged with severity (Critical/High/Medium/Low) and tracked to closure.
- Critical/High defects block go-live; Medium/Low may be deferred with sponsor approval.

12. Approval & Sign-Off

This Scope of Work becomes the project's blueprint upon written approval below. Any material change to scope, data source, timeline, or technology stack after approval should be handled through a change request.

Approved by (Jalab khan): _____ Date: _____